

Applied Statistical Programming

Week 2

Evaluating Code Performance

When evaluating code runtime, there are three important questions:

- ❶ **Problem size:** How many observations, parameters, iterations, or comparisons are required?
- ❷ **Algorithmic complexity:** Does the work grow with n ? If so, at what rate?
- ❸ **Implementation costs:** How much time is added by each component of the code?

What grows with n ?

Let $T(n)$ denote the number of elementary operations required for an input of size n .

Big- O gives an asymptotic **upper bound**. We write We write

$$T(n) = O(g(n))$$

if there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$0 \leq T(n) \leq c g(n) \quad \text{for every } n \geq n_0.$$

Thus, $T(n) = O(g(n))$ means that, beyond some input size, $T(n)$ is bounded above by a fixed multiple of $g(n)$.

What grows with n ?

Note that:

- $O(\cdot)$ is an upper bound, not necessarily a tight bound.
- The tighter statement is $T(n) = \Theta(n)$.

Big- Θ gives the exact **asymptotic order**. We write $T(n) = \Theta(g(n))$ if there exist $c_1, c_2 > 0$ and n_0 such that

$$c_1 g(n) \leq T(n) \leq c_2 g(n) \quad \text{for every } n \geq n_0.$$

Example 1: Linear search in an array

Suppose we search an unsorted array for a target value.

```
linear_search <- function(x, target) {  
  for (i in seq_along(x)) {  
    if (x[i] == target) return(i)  
  }  
  return(NA_integer_)  
}
```

For an array of length n :

- Best case: the target is first, so $T_{\text{best}}(n) = 1$.
- Worst case: the target is last or absent, so $T_{\text{worst}}(n) = n$
- If the target is equally likely to take any position, then

$$\mathbb{E}[T(n)] = \frac{1}{n} \sum_{i=1}^n i = \frac{n(n+1)}{2n} = \frac{n+1}{2}.$$

- In this case, we say $T(n) = \Theta(n)$

Example 2: Binary search

If the array is sorted, binary search compares the target with the middle element and discards half of the remaining array.

After k comparisons, the number of possible elements remaining is at most: $\frac{n}{2^k}$

The algorithm stops when this quantity is at most 1: $\frac{n}{2^k} \leq 1$

Solving for k gives

$$n \leq 2^k \iff \log_2 n \leq k.$$

More precisely, the worst-case number of comparisons is: $T(n) = \lfloor \log_2 n \rfloor + 1$

Hence:

$$T(n) = \Theta(\log n).$$

- Searching $1,024 = 2^{10}$ elements requires at most 11 comparisons.
- Searching $1,048,576 = 2^{20}$ elements requires at most 21.

Example 3: Bubble sort

Bubble sort repeatedly compares adjacent elements and swaps those that are out of order.

```
for (i in seq_len(n - 1)) {  
  for (j in seq_len(n - i)) {  
    if (x[j] > x[j + 1]) {  
      temp <- x[j]  
      x[j] <- x[j + 1]  
      x[j + 1] <- temp  
    }  
  }  
}
```

The first pass makes $n - 1$ comparisons, the second makes $n - 2$, and so on:

$$T(n) = (n - 1) + (n - 2) + \cdots + 1 = \sum_{j=1}^{n-1} j.$$

Using the formula for an arithmetic series,

$$T(n) = \frac{n(n - 1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n.$$

Therefore, we say $T(n) = \Theta(n^2)$

Why lower-order terms do not “matter”?

Consider

$$T(n) = 3n^2 + 20n + 100.$$

To prove that $T(n) = O(n^2)$, we must find constants $c > 0$ and n_0 such that

$$3n^2 + 20n + 100 \leq cn^2 \quad \text{for every } n \geq n_0.$$

For every $n \geq 1$,

$$n \leq n^2 \quad \text{and} \quad 1 \leq n^2.$$

Therefore,

$$3n^2 + 20n + 100 \leq 3n^2 + 20n^2 + 100n^2 = 123n^2.$$

Taking $c = 123$ and $n_0 = 1$ we have that

$$T(n) = O(n^2).$$

Moreover,

$$T(n) \geq 3n^2$$

for every $n \geq 1$. As a result,

$$T(n) = \Theta(n^2).$$

The terms $20n$ and 100 matter for actual running time, but they do not change the asymptotic order.

Why multiplicative constants do not “matter”?

Suppose we need to process a dataset with n observations.

- Algorithm A performs about 5 operations per observation:

$$T_A(n) = 5n.$$

- Algorithm B performs about 500 operations per observation:

$$T_B(n) = 500n.$$

Algorithm B is much slower: for the same dataset, it does 100 times as much work.

Why multiplicative constants do not “matter”?

Now ask yourself: what happens when the dataset doubles in size?

	n	$2n$
T_A	$5n$	$10n$
T_B	$500n$	$1000n$

For *both* algorithms, doubling the input doubles the work.

$$T_A(n) = \Theta(n) \quad \text{and} \quad T_B(n) = \Theta(n).$$

The constant clearly affects how fast the algorithm is, but not how its running time scales as the problem gets larger.

Comparing common orders of complexity

To sum,

$$1 \prec \log n \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n,$$

where $f(n) \prec g(n)$ means

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0.$$

Two important rules

To simplify complexity calculations, note that:

- **Sum rule:** If two sequential steps cost $O(f(n))$ and $O(g(n))$, their total cost is

$$O(f(n) + g(n)) = O(\max\{f(n), g(n)\}).$$

- **Product rule:** If an outer loop runs $O(f(n))$ times and each iteration costs $O(g(n))$, the total cost is

$$O(f(n)g(n)).$$

Vectorization

Vectorization means that a piece of code has been carefully engineered to run as efficiently as possible.

Suppose we want $z_i = (x_i - \bar{x})/s_x$. In vector form:

$$\mathbf{z} = \frac{\mathbf{x} - \bar{x}\mathbf{1}}{s_x}.$$

```
# Explicit R loop
z_loop <- numeric(length(x))
for (i in seq_along(x)) z_loop[i] <- (x[i] - mean(x)) / sd(x)

# Vectorized: mean/sd computed once
mu <- mean(x); sx <- sd(x)
z_vec <- (x - mu) / sx
```

The first version is not efficient because it recomputes $\bar{x}$ and s_x in every iteration.

Loops are not the enemy

Sometimes the next step of an algorithm depends on the result of the previous step:

$$\theta^{(1)} \longrightarrow \theta^{(2)} \longrightarrow \theta^{(3)} \longrightarrow \dots$$

For example,

$$\theta^{(t+1)} = G(\theta^{(t)}).$$

- This is exactly how algorithms such as EM, MCMC, and coordinate descent work.
- In these cases, a loop is the natural way to express the computation: compute the current step, update the state, and continue.
- The important performance question is not “Can I remove the for loop?”
- Instead ask: “Is the expensive work *inside* the loop being done efficiently?”

Functional iteration

Now consider a different situation: each iteration is independent.

- Suppose we want to run the same bootstrap calculation B times
- Each bootstrap replication can be described as one self-contained function

```
# Observed data
x <- c(12, 15, 11, 18, 14, 16, 13, 17)

# Number of bootstrap replications
B <- 1000

set.seed(123)

# Perform one bootstrap replication
boot_mean <- function(i, data) {
  resample <- sample(data, size = length(data), replace = TRUE)
  mean(resample)
}

# Repeat the independent calculation B times
boot_estimates <- vapply(X = seq_len(B), FUN = boot_mean, FUN.VALUE = numeric(1), data = x)
```

The important idea is not that `vapply()` avoids a loop. What matters is that we have clearly defined:

one iteration = one independent unit of work

Memory is part of performance

Code can be computationally simple and still fail because the objects it creates are too large.

A numeric value stored as a double typically requires 8 bytes. Therefore, a dense $n \times p$ matrix requires approximately $8np$ bytes.

For example, a $50,000 \times 50,000$ matrix contains 2.5 billion entries.

At 8 bytes per entry, that is already about 20 GB

Memory is part of performance

How to overcome this issue? For *one* matrix, we have that large intermediate objects can therefore be more expensive than the final result.

- If we only need a small fraction of all possible entries, do not construct the full object in the first place.
- For example, in record linkage, storing every pairwise comparison can be wasteful when most record pairs are obviously impossible matches.
- Sparse representations store only the entries that matter, reducing storage from $O(np)$ to roughly $O(\text{nnz})$, where nnz is the number of nonzero or stored entries.
- Warning: some transformations also create temporary copies, so an operation on a 20 GB object may briefly require substantially more than 20 GB of RAM.

Numerical stability

We work on the premise that oftentimes, mathematically equal is not computationally equal.

For large a, b , direct evaluation of

$$\log(e^a + e^b)$$

can overflow even when the final log is representable. Let $m = \max(a, b)$:

$$\log(e^a + e^b) = m + \log(e^{a-m} + e^{b-m}).$$

This **log-sum-exp** identity will be essential for mixture models and variational inference.

Refactoring

Refactoring in coding means that we preserve the end goal, but change the structure of the code

Let f_0 be the baseline and f_1 the refactor. For a domain $\mathcal{X}_T$, preservation means

$$f_1(x) \approx f_0(x) \quad \forall x \in \mathcal{X}_T,$$

with tolerance chosen from the method's numerical scale.

- For deterministic methods, equality may be exact.
- For approximations, order changes can produce tiny differences.
- For randomized algorithms, compare distributions or fixed-seed outputs as appropriate.

Example

```
# Baseline: recomputes differences repeatedly
sqdist_loop <- function(X, center) {
  out <- numeric(nrow(X))
  for (i in seq_len(nrow(X))) {
    out[i] <- sum((X[i, ] - center)^2)
  }
  out
}

# Vectorized by rows
sqdist_vec <- function(X, center) {
  Z <- sweep(X, 2, center, '-')
  rowSums(Z * Z)
}
```

Improve functions need a baseline

Before accepting an optimization:

$$\underbrace{\max_i |f_1(x_i) - f_0(x_i)|}_{\text{discrepancy}} \leq \delta \quad \text{then evaluate} \quad \underbrace{\frac{T_0}{T_1}}_{\text{speedup}}.$$

- Choose δ from substantive/numerical requirements, not convenience.
- It is important to benchmark only after the new implementation passes correctness tests.
- Record both discrepancy and speedup in the benchmark report e.g., a log file